

 Powered by Arizona State University®	Universidad Internacional del Ecuador	
	Escuela de Ciencias de la Computación Carrera Ingeniería en Sistemas de Información	
Nombre: Mike Quijije Chele		Trabajo: Autónomo / Práctico
Materia: ARQUITECTURA DE COMPUTADORAS Y SISTEMAS OPERATIVOS		Semestre: Primero
Fecha: 19/8/2025		Profesor: Wernher Téllez Gómez

1. Descripción de la asignación

El presente documento constituye la evidencia de desarrollo del desafío MiniTienda, un sistema de consola para el registro y análisis de ventas de una tienda de tecnología. El programa implementa un catálogo de productos, registro de ventas con validaciones, persistencia en archivos CSV, cálculo de métricas estadísticas con NumPy, visualización de datos con Matplotlib y un menú interactivo controlado por un bucle while.

2. Estructuras de datos utilizadas

A continuación se detalla qué estructura de datos de Python se usó en cada parte del sistema, tal como lo solicita la guía de la asignación:

Estructura	Variable / uso en el código	Justificación
Tupla	CATALOGO = ("Laptop Dell XPS", ...)	Los nombres de los productos no cambian durante la ejecución normal, por lo que se usa una tupla (inmutable) como catálogo base.
Diccionario	PRECIOS = {0: 1200.00, ...} STOCK = {0: 10, ...}	Permiten acceder al precio o stock de un producto en tiempo O(1) usando su ID como clave.
Lista	ventas_buffer = []	Almacena cada venta (un diccionario por venta) en el orden en que se registran; permite agregar elementos dinámicamente con .append().

Nota: al agregar un nuevo producto (Reto A), como las tuplas son inmutables, se reconstruye —CATALOGO = CATALOGO + (nombre,)— en lugar de modificarla in-place, respetando la naturaleza de la tupla.

3. ¿Qué parte se hizo con Pandas y qué parte con NumPy?

REINVENTEMOS
EL FUTURO

Pandas

- `guardar_csv()` / `cargar_csv()`: convierten el `ventas_buffer` (lista de diccionarios) a un `DataFrame` con `pd.DataFrame()` y lo exportan/importan con `to_csv()` / `read_csv()`.
- `analizar_ventas()`: construye un `DataFrame` a partir de `ventas_buffer` y usa `df.groupby('producto')['total'].sum()` para calcular los ingresos por producto, y `df.groupby('producto')['cantidad'].sum().idxmax()` para hallar el producto más vendido.
- `graficar_ingresos()` / `exportar_grafico_png()`: usan el resultado de ese `groupby` de `pandas` como fuente de datos para el gráfico de barras.

NumPy

- En `analizar_ventas()`, la columna 'total' del `DataFrame` se convierte a un arreglo con `np.array(df['total'])`.
- Sobre ese arreglo se calculan las métricas estadísticas: `np.sum()` (ingresos totales), `np.mean()` (ingreso promedio) y `np.std()` (desviación estándar de los montos vendidos).

En resumen: Pandas se encarga de estructurar, agrupar y persistir los datos (DataFrame, groupby, CSV), mientras que NumPy se usa puntualmente para los cálculos numéricos agregados sobre el arreglo de totales.

4. ¿Dónde se usó try/except y por qué?

Función	Qué protege	Por qué
<code>registrar_venta()</code>	Conversión de ID/cantidad a int, validación de existencia en catálogo, validación de cantidad positiva y de stock disponible.	El usuario puede escribir texto no numérico, un ID que no existe, una cantidad negativa o mayor al stock; el <code>try/except/else/finally</code> evita que el programa se caiga y además registra el error en <code>log.txt</code> .
<code>guardar_csv()</code> / <code>cargar_csv()</code>	Escritura y lectura del archivo <code>ventas.csv</code> .	El archivo puede no existir aún (primera ejecución) o el disco puede fallar al escribir; se captura <code>FileNotFoundError</code> y <code>Exception</code> genérica.
<code>analizar_ventas()</code>	Cálculo del descuento promedio (<code>ventas_con_descuento.sum()</code> / <code>len(...)</code>).	Si ninguna venta tuvo descuento, <code>len(...)</code> es 0 y la división lanzaría <code>ZeroDivisionError</code> ; se captura explícitamente para informarlo sin romper el programa (división por cero controlada).
<code>graficar_ingresos()</code> / <code>exportar_grafico_png()</code>	Generación y guardado del gráfico con <code>Matplotlib</code> .	Evita que un error al graficar (por ejemplo, sin datos) detenga el resto del sistema.

REINVENTEMOS
EL FUTURO

agregar_producto()	Conversión de precio/stock a float/int y validación de valores negativos.	El usuario podría ingresar texto no numérico o valores negativos para el precio o el stock.
menu()	Conversión de la opción elegida a int, y el bucle completo.	Controla entradas no numéricas, Ctrl+C (KeyboardInterrupt) y cualquier error no previsto en las funciones llamadas desde el menú.

El caso más completo es registrar_venta(), que implementa la estructura try / except / else / finally solicitada en la guía: el bloque else solo se ejecuta si la venta pasó todas las validaciones (ahí se descuenta el stock y se guarda la venta), y el bloque finally siempre se ejecuta al final, haya habido error o no, para indicar que la operación terminó.

5. Explicación breve del algoritmo (ciclos y validaciones)

El programa se organiza alrededor de un bucle while True en la función menu(), que se repite indefinidamente mostrando las opciones disponibles hasta que el usuario elige la opción 0, momento en el que se ejecuta un break para salir del ciclo. Si el usuario no ingresa nada, un continue vuelve a mostrar el menú sin procesar ninguna opción.

Dentro del menú se usa una cadena de if / elif / else para decidir qué función ejecutar según la opción numérica ingresada, y un try/except adicional protege la conversión de esa opción a entero y cualquier excepción no controlada dentro de las funciones llamadas.

Las validaciones de negocio (ID inexistente, cantidad inválida, stock insuficiente, división por cero al calcular el descuento promedio) se implementan como excepciones controladas (raise ValueError(...)) que son capturadas más arriba en la misma función, evitando que un dato mal ingresado detenga la ejecución del programa. Cada vez que ocurre un error de este tipo durante el registro de una venta, se deja constancia en el archivo log.txt junto con la fecha y hora exacta (Reto D).

6. Retos implementados

Reto	Descripción	Estado
A	Agregar un producto nuevo al catálogo y actualizar precios/stock.	Implementado — función agregar_producto(), opción 8.
B	Exportar el gráfico a PNG con plt.savefig('ingresos.png').	Implementado — función exportar_grafico_png(), opción 7.
C	Aplicar descuento del 5% si la cantidad vendida es ≥ 10 unidades.	Implementado — bloque if cantidad ≥ 10 dentro de registrar_venta().

REINVENTEMOS
EL FUTURO

D	Validar que no se pueda vender un producto_id inexistente y registrar el intento fallido en log.txt.	Implementado — validación con raise ValueError y escritura en log.txt dentro del except.
---	--	--

Nota sobre numeración: en la guía, el Reto B sugiere ubicar la exportación a PNG como opción “6”. En el programa entregado, la opción 6 se usa para “Mostrar gráfica” (en pantalla) y la exportación a PNG quedó como opción 7, para no eliminar la visualización en pantalla. Funcionalmente el Reto B está completo (usa exactamente `plt.savefig("ingresos.png")`); si se requiere que sea estrictamente la opción 6, basta con intercambiar el orden de esas dos opciones en el menú.

7. Evidencia de ejecución del programa

Las siguientes capturas corresponden a una ejecución real del programa `minitienda.py`, tomadas en secuencia dentro de una misma sesión de consola, mostrando entradas y salidas.

7.1 Menú principal y catálogo (tuplas + diccionarios)

The screenshot shows the Visual Studio Code interface. On the left, the Explorer panel shows a file named `minitienda.py` selected. The main editor area displays the output of running the program in a terminal. The program starts with a welcome message and presents a menu of options for the 'MINITIENDA - SISTEMA DE VENTAS'. The options are:

- 1) Ver catálogo
- 2) Registrar venta
- 3) Guardar ventas (CSV)
- 4) Cargar ventas (CSV)
- 5) Analizar ventas
- 6) Mostrar gráfica
- 7) Exportar gráfica a PNG (RETO B)
- 8) Agregar producto (RETO A)
- 9) Generar datos de ejemplo
- 0) Salir

The prompt 'Seleccione una opción:' is visible at the bottom of the menu. On the right side of the VS Code window, there is a 'Chat' panel with a message from an AI agent: 'Compilar con agente. Las respuestas de IA pueden ser inexactas. Generar instrucciones de agente para incorporar inteligencia artificial en el código base.'

7.2 Registro de venta exitosa

The screenshot shows a VS Code window with a Python file named `minitienda.py` open. The terminal output displays the following:

```
=====
Seleccione una opción: 2

=====
ID          PRODUCTO          PRECIO    STOCK
=====
0  Laptop Dell XPS          $ 1200.00    10
1  Mouse Logitech          $   45.50    50
2  Teclado Mecánico        $   89.90    30
3  Monitor Samsung        $  350.00    15
4  Audifonos Sony          $   120.00    25
=====

ID del producto: 3
Cantidad: 5

Venta registrada: Monitor Samsung x5 = $1750.00
Operación de registro de venta finalizada.

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 1
```

The right sidebar shows the 'Chat' panel with the 'minitienda.py' agent, indicating that the AI can generate instructions for incorporating artificial intelligence into the code base.

7.3 Manejo de errores y registro en log.txt (Reto D)

The screenshot shows the same VS Code window as before, but now the terminal output displays an error message:

```
=====
Operación de registro de venta finalizada.

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 2

=====
ID          PRODUCTO          PRECIO    STOCK
=====
0  Laptop Dell XPS          $ 1200.00    10
1  Mouse Logitech          $   45.50    50
2  Teclado Mecánico        $   89.90    30
3  Monitor Samsung        $  350.00    10
4  Audifonos Sony          $   120.00    25
=====

ID del producto: 45

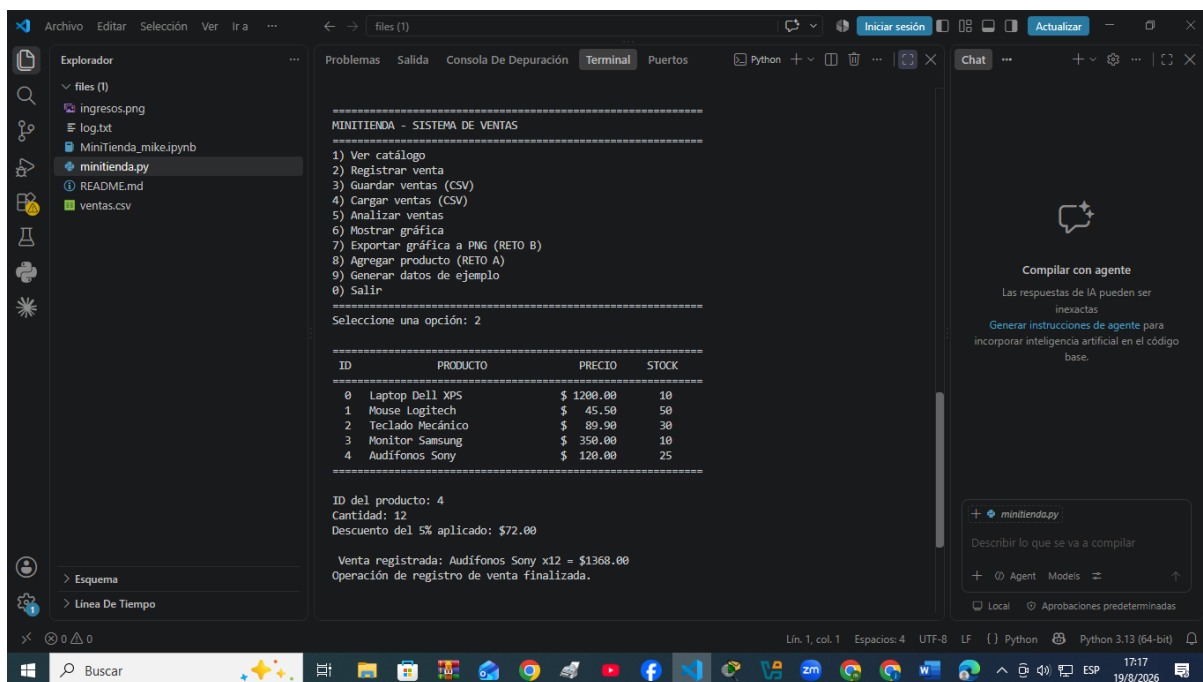
Error: Producto ID 45 no existe en el catálogo
Operación de registro de venta finalizada.

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 2
```

The error message indicates that the product ID 45 does not exist in the catalog, demonstrating error handling in the application.

**REINVENTEMOS
EL FUTURO**

7.4 Descuento por volumen ≥ 10 unidades (Reto C)



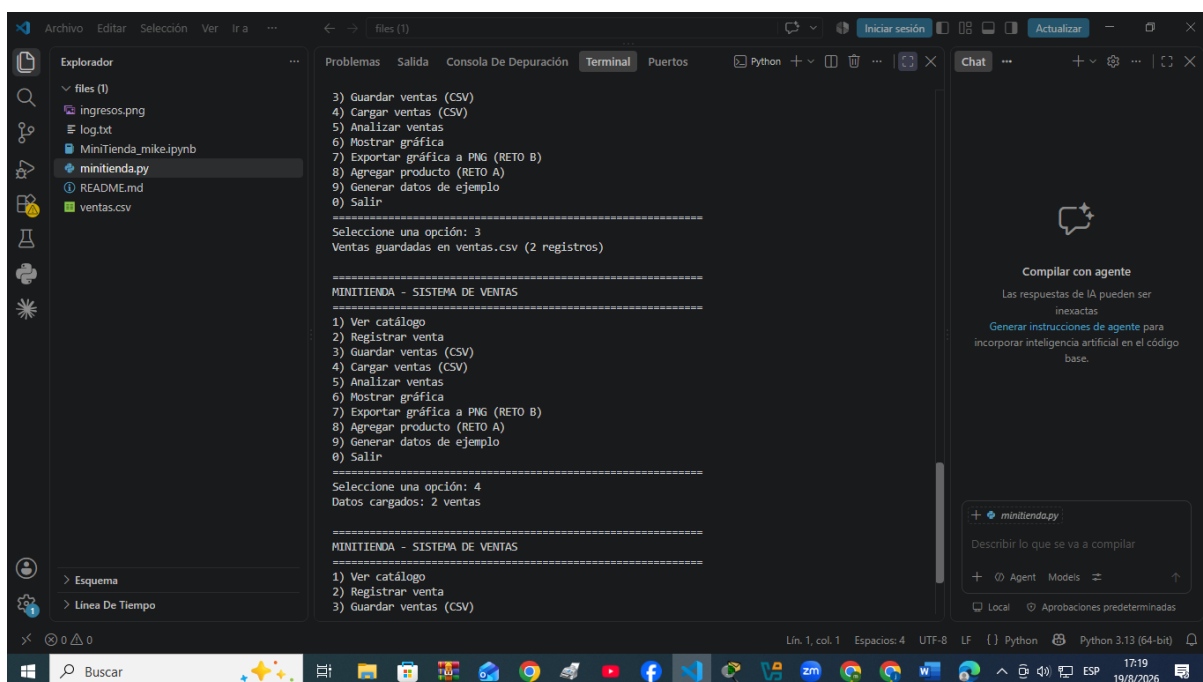
```
=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 2

=====
ID          PRODUCTO          PRECIO    STOCK
=====
0  Laptop Dell XPS          $ 1200.00    10
1  Mouse Logitech           $ 45.50     50
2  Teclado Mecánico         $ 80.90     30
3  Monitor Samsung          $ 350.00    10
4  Audifonos Sony           $ 120.00    25
=====

ID del producto: 4
Cantidad: 12
Descuento del 5% aplicado: $72.00

Venta registrada: Audifonos Sony x12 = $1368.00
Operación de registro de venta finalizada.
```

7.5 Guardar y cargar ventas en CSV (pandas)



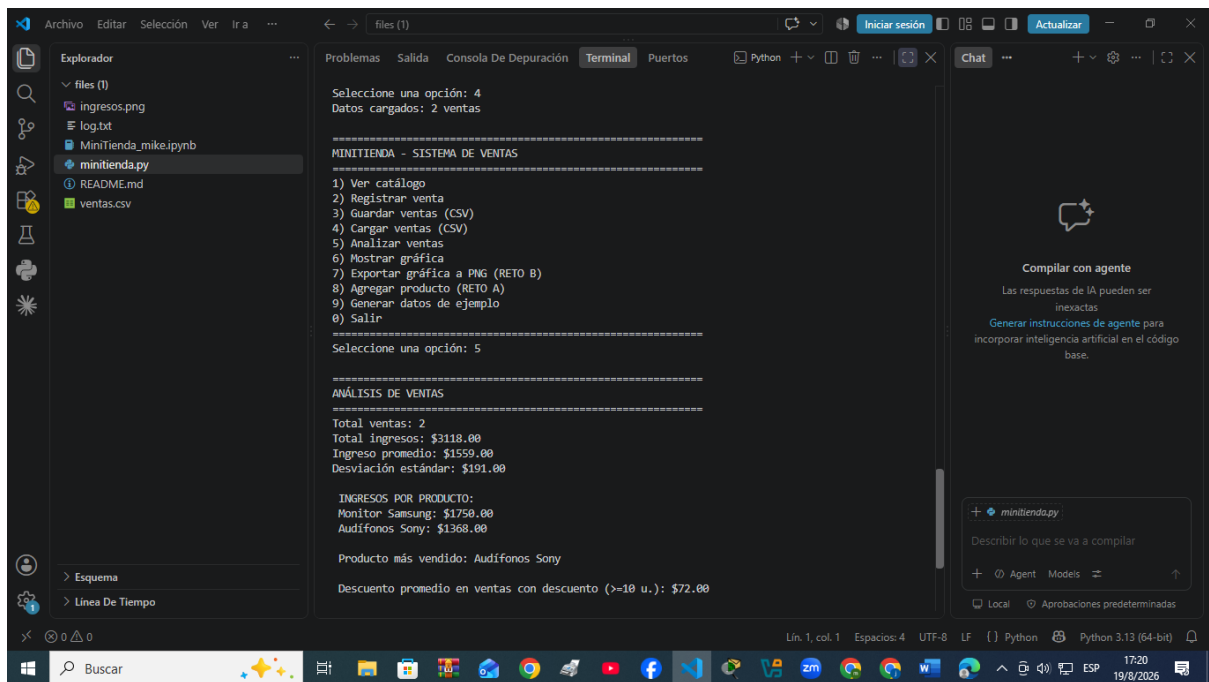
```
=====
MINITIENDA - SISTEMA DE VENTAS
=====
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 3
Ventas guardadas en ventas.csv (2 registros)

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 4
Datos cargados: 2 ventas

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
```

**REINVENTEMOS
EL FUTURO**

7.6 Análisis de ventas (NumPy + pandas)



```
Archivo  Editar  Selección  Ver  Ir a  ...  files (1)  Iniciar sesión  Actualizar

Explorador
  files (1)
  ingresos.png
  log.txt
  MiniTienda_mike.ipynb
  minitienda.py
  README.md
  ventas.csv

Problemas  Salida  Consola De Depuración  Terminal  Puertos  Python +  Python 3.13 (64-bit)  Chat ...

Seleccione una opción: 4
Datos cargados: 2 ventas

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 5

=====
ANÁLISIS DE VENTAS
=====
Total ventas: 2
Total ingresos: $3118.00
Ingreso promedio: $1559.00
Desviación estándar: $191.00

INGRESOS POR PRODUCTO:
Monitor Samsung: $1750.00
Audífonos Sony: $1368.00

Producto más vendido: Audífonos Sony

Descuento promedio en ventas con descuento (>=10 u.): $72.00

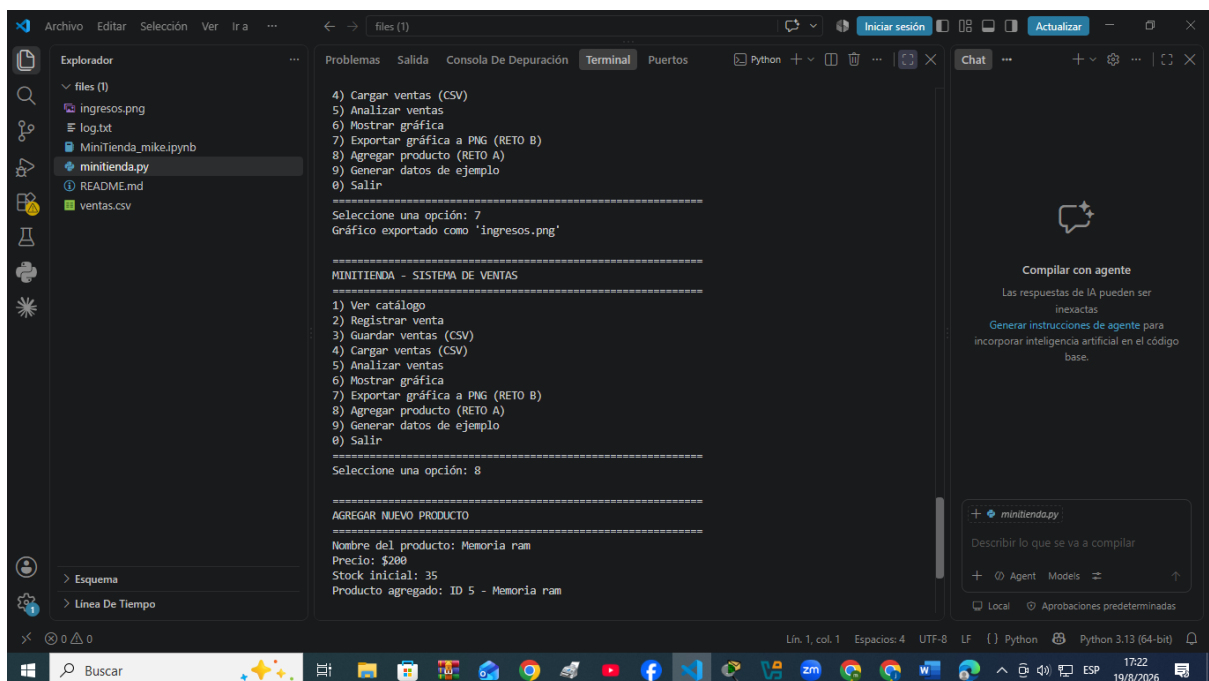
=====

Compilar con agente
Las respuestas de IA pueden ser inexactas.
Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

+ minitiendapy
Describir lo que se va a compilar
+ Agente Models
Local Aprobaciones predeterminadas

Lín. 1, col. 1  Espacios: 4  UTF-8  LF  Python  Python 3.13 (64-bit)
```

7.7 Exportar gráfico a PNG (Reto B), agregar producto (Reto A) y generar datos de ejemplo



```
Archivo  Editar  Selección  Ver  Ir a  ...  files (1)  Iniciar sesión  Actualizar

Explorador
  files (1)
  ingresos.png
  log.txt
  MiniTienda_mike.ipynb
  minitienda.py
  README.md
  ventas.csv

Problemas  Salida  Consola De Depuración  Terminal  Puertos  Python +  Python 3.13 (64-bit)  Chat ...

4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 7
Gráfico exportado como 'ingresos.png'

=====
MINITIENDA - SISTEMA DE VENTAS
=====
1) Ver catálogo
2) Registrar venta
3) Guardar ventas (CSV)
4) Cargar ventas (CSV)
5) Analizar ventas
6) Mostrar gráfica
7) Exportar gráfica a PNG (RETO B)
8) Agregar producto (RETO A)
9) Generar datos de ejemplo
0) Salir
=====
Seleccione una opción: 8

=====
AGREGAR NUEVO PRODUCTO
=====
Nombre del producto: Memoria ram
Precio: $200
Stock inicial: 35
Producto agregado: ID 5 - Memoria ram

=====
Seleccione una opción: 9

=====
GENERAR DATOS DE EJEMPLO
=====
Datos generados exitosamente.

=====

Compilar con agente
Las respuestas de IA pueden ser inexactas.
Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

+ minitiendapy
Describir lo que se va a compilar
+ Agente Models
Local Aprobaciones predeterminadas

Lín. 1, col. 1  Espacios: 4  UTF-8  LF  Python  Python 3.13 (64-bit)
```

**REINVENTEMOS
EL FUTURO**